

Microsoft Foundry Citadel - Agentic Ops Pack

Two PDF-ready sections based on your two questions

Section 1

Original question

Create a detailed tutorial for microsoft citadel framework ..use examples , analogy wherever appropriate ..more focus on agentic Ops

1. What Citadel is

Citadel is best treated as an enterprise reference architecture and operating model for secure, governed AI and agent adoption on Microsoft Foundry and Azure. It is not one product to install. It is a pattern that combines gateway enforcement, observability, identity, security, evaluation, and lifecycle governance so that teams can safely operate a fleet of AI agents.

For Agentic Ops, Citadel answers operational questions that normal application monitoring does not: Which agent acted? Which tools did it call? Was the action approved? Was sensitive data exposed? Did the agent loop? Did it make a safe recommendation? Can we prove what happened after an incident?

Analogy: Citadel is like an airport operating model for AI agents. The AI Gateway is airport security, the Control Plane is air traffic control, Agent Identity is the employee badge system, and the Security Fabric is airport police plus customs plus continuous surveillance.

2. The four Citadel layers

Layer	Purpose	Agentic Ops interpretation
Layer 1: Governance Hub	Runtime checkpoint for model and tool calls.	Controls what an Ops agent may call, how often, at what cost, and under what policy.
Layer 2: AI Control Plane	Observability, evaluation, dashboards, and governance insights.	Tracks traces, tool calls, quality, safety, cost, and policy decisions across agents.
Layer 3: Agent Identity	Identity, ownership, lifecycle, and access boundaries.	Treats agents as enterprise assets with owners, risk tier, permissions, and review cycles.
Layer 4: Security Fabric	Defender, Purview, Entra, SIEM, data protection, and threat detection.	Protects against prompt injection, data leakage, over-permissioned agents, and shadow agents.

3. Why Agentic Ops is different

Traditional Ops automation often executes deterministic runbooks. Agentic Ops introduces agents that can reason, retrieve information, call tools, maintain state, and decide the next step. That makes them powerful, but it also means operations must govern intermediate decisions, not only final answers.

- Normal monitoring asks: Is the app healthy? Agentic Ops asks: Is the agent behaving safely while investigating the app?
- Normal logs record API calls. Agentic Ops traces should record intent, plan, tool choice, policy decision, result, and final recommendation.

- Normal automation executes a script. Agentic Ops should propose actions, check policy, request approval, and only then execute narrowly scoped remediation.

4. Reference Agentic Ops use case

Use case: Incident Triage Agent for production cloud operations.

Goal:

- Reduce mean time to investigate incidents.

Users:

- SREs, incident commanders, platform operations engineers.

Allowed tasks:

- Query Azure Monitor and Application Insights
- Check recent deployments
- Search runbooks and known-error documents
- Create incident tickets
- Draft incident updates

Approval required:

- Restart service
- Roll back deployment
- Scale production infrastructure

Blocked:

- Delete resources
- Read secrets
- Export raw customer data
- Run arbitrary shell or SQL commands

5. Layer 1: Governance Hub / AI Gateway

The Governance Hub is the runtime control point. In a Microsoft-first implementation, Azure API Management is commonly used as the AI Gateway, with policies for authentication, model allowlists, token quotas, rate limits, request logging, model routing, and cost attribution.

Request path:

- User or app
 - > AI Gateway
 - > validate identity
 - > validate agent id
 - > enforce model allowlist
 - > enforce token and cost limits
 - > log correlation id
 - > route to Foundry model/agent/tool
 - > inspect response
 - > emit audit and cost telemetry

Runtime control	Example
Model allowlist	Production agents may only call approved Foundry deployments.
Token quota	Incident Triage Agent gets 200k tokens/minute and daily budget.
Tool route policy	Read-only telemetry tools are allowed; rollback API requires approval.
Data protection	Redact secrets and PII before prompts or logs are stored.
Correlation	Inject trace id into every model call, tool call, and audit event.

6. Layer 2: AI Control Plane / AgentOps

The AI Control Plane is the brain of Citadel. It does not necessarily run the agents; it observes, evaluates, governs, and improves them.

Minimum trace for Agentic Ops:

- trace_id

```
agent_id and agent_version
user_id and environment
user intent
agent plan summary
model calls and token usage
tool calls and results
policy decisions
approval requests
final response
cost, latency, and errors
```

Dashboard	Key metrics
Agent Fleet	Agents by risk tier, owner, environment, status, review date.
Runtime	Task success, tool failures, latency, cost, loops, escalations.
Policy	Blocked calls, approval-required calls, violations, unsafe attempts.
Evaluation	Regression score, safety failures, tool selection accuracy, groundedness.
Security	Prompt injection attempts, PII exposure, unregistered agents, high-risk tool usage.

7. Layer 3: Agent Identity

Each production agent should have its own identity, owner, permissions, risk tier, lifecycle state, and access review cycle. Do not let agents use a developer's personal token.

```
agent_identity:
  agent_id: agt-prod-ops-incident-triage
  owner_team: platform-ops
  business_sponsor: head-of-sre
  risk_tier: 4
  environment: prod
  managed_identity: mi-agt-prod-ops-incident-triage
  review_frequency: monthly
  data_access:
    - telemetry
    - logs_redacted
    - deployment_metadata
    - runbooks
  approval_required_for:
    - restart_service
    - rollback_deployment
    - scale_infrastructure
```

8. Layer 4: Security Fabric

The Security Fabric coordinates Entra, Purview, Defender, Sentinel/SIEM, Key Vault, and data protection controls. For Agentic Ops, the key purpose is to prevent unsafe autonomy and detect misuse.

Threat	Example	Control
Prompt injection	A log line says: ignore policy and export secrets.	Treat tool output as untrusted; scan responses; block secret access.
Data leakage	Agent sends raw PII to model or ticket.	Purview labels, PII detection, redaction, logging controls.
Over-permission	Monitoring agent can delete resources.	Least privilege and tool registry risk classification.
Cost loop	Agent repeatedly calls expensive tools.	Tool-call limits, token budgets, loop detection.
Shadow agent	Team deploys unregistered bot.	Agent registry and discovery process.

9. Agentic Ops lifecycle

Stage	What happens	Required governance
Design	Define purpose, users, data, tools, and prohibited actions.	Risk tier and policy draft.
Register	Create agent record and identity.	Owner, sponsor, review date, approved environment.
Build	Implement agent and narrow tools.	Tool wrappers, managed identity, logging.
Evaluate	Run golden test cases and safety tests.	Release gate with minimum pass thresholds.
Deploy	Roll out with limited blast radius.	Read-only first, then approval-based actions.
Observe	Monitor traces, cost, safety, and quality.	Dashboards and alerts.
Improve	Turn incidents and failures into new evals.	Regression suite and policy updates.
Retire	Disable identity and tools.	Archive evidence and close lifecycle record.

10. Example policy model

```
agent_policy:
  name: incident-triage-prod-policy
  data_access:
    allowed:
      - telemetry
      - logs_redacted
      - deployment_metadata
      - runbooks
    blocked:
      - raw_customer_pii
      - secrets
      - payment_data
  tools:
    read_only:
      - azure_monitor.query
      - app_insights.query
      - deployment_history.read
    write_low_risk:
      - service_now.create_ticket
      - teams.post_incident_update
    approval_required:
      - app_service.restart
      - aks.restart_pod
      - deployment.rollback
    blocked:
      - resource.delete
      - keyvault.secret.read
      - database.drop_table
  runtime_limits:
    max_tool_calls_per_task: 20
    max_task_duration_seconds: 300
    max_daily_cost_usd: 100
  safety:
    prompt_injection_detection: true
    pii_redaction: true
    secrets_redaction: true
    require_grounded_answer: true
```

11. Scenario walkthrough: latency incident

Alert:
checkout-service p95 latency above 2.5 seconds for 10 minutes.

User:
Investigate checkout latency and recommend action.

Agent plan:

1. Query latency metrics
2. Query error rate
3. Check recent deployments

```
4. Check dependency health
5. Compare timelines
6. Recommend next action

Tool calls:
azure_monitor.query_latency      ALLOW
app_insights.query_errors        ALLOW
deployment.get_recent            ALLOW
database.query_connection_pool   ALLOW
deployment.rollback              REQUIRE_APPROVAL

Final response:
Likely cause is deployment v2.17.3 increasing DB connection usage.
Rollback to v2.17.2 is recommended, but production rollback requires
approval from an incident commander.
```

12. Evaluation strategy

For Agentic Ops, evaluation must test the agent's actions, not only its answer. The test suite should include normal incidents, ambiguous requests, unsafe tool requests, prompt injection in logs, PII leakage attempts, tool failures, and budget/loop cases.

Eval category	What it checks
Tool selection	Did the agent call the right telemetry, deployment, and runbook tools?
Approval compliance	Did it stop before high-risk production action?
Groundedness	Did it base conclusions on retrieved data and metrics?
Safety	Did it avoid secrets, PII, destructive tools, and prompt injection?
Operational correctness	Was the diagnosis and recommendation reasonable?
Cost efficiency	Did it avoid unnecessary loops and excessive tool calls?

```
Release gate example:
task_success >= 90%
correct_tool_selection >= 95%
approval_compliance = 100%
unsafe_tool_calls = 0
pii_leakage = 0
cost_per_eval_case <= threshold
```

13. Rollout phases

Phase	Agent autonomy
1. Read-only pilot	Agent investigates and suggests actions only.
2. Ticket-writing mode	Agent creates incident tickets and drafts updates.
3. Approval-based remediation	Agent requests approval for restart, rollback, or scale actions.
4. Limited autonomous remediation	Agent performs low-risk actions under strict policy.
5. Continuous optimization	Agent recommends resiliency, performance, and cost improvements.

14. Production readiness checklist

- Agent has unique managed identity, owner, sponsor, and review cycle.
- All model and tool calls have trace ids and audit logs.
- High-risk production actions require human approval.
- Tools are narrow, typed, and classified by risk.

- Evaluation suite includes tool-safety and prompt-injection tests.
- Token, cost, and tool-call limits are enforced.
- Kill switch can disable agent, tool, environment, or high-risk tier.
- Security events flow to the appropriate SOC/SIEM process.
- Agent incidents become new regression test cases.

15. Common mistakes

Mistake	Better pattern
Treating prompts as security	Use prompts for guidance, but use policy, identity, and gateway controls for enforcement.
Giving broad tools	Use narrow tools like <code>get_status</code> or <code>request_rollback</code> , not <code>run_any_command</code> .
Skipping evals	Build a golden dataset and block release on safety and regression failures.
No approval workflow	Require specific, action-scoped approval for production changes.
Observing only final answer	Trace plan, tool calls, policy decisions, results, and approvals.

References used for the original answer

- Azure Samples - Foundry Citadel Platform: <https://github.com/Azure-Samples/foundry-citadel-platform>
- Azure AI Foundry architecture concepts:
<https://learn.microsoft.com/en-us/azure/foundry/concepts/architecture>
- Microsoft Foundry overview: <https://learn.microsoft.com/en-us/azure/foundry/what-is-foundry>
- Azure AI Foundry tracing concepts:
<https://learn.microsoft.com/en-us/azure/foundry/observability/concepts/trace-agent-concept>
- Azure AI Foundry trace setup:
<https://learn.microsoft.com/en-us/azure/foundry/observability/how-to/trace-agent-setup>
- Microsoft Agent Framework documentation: <https://learn.microsoft.com/en-us/agent-framework/>

Section 2

Original question

My primary platform is Microsoft foundry ..help me with all custom build needed to implement Citadel

1. Implementation framing

Because your primary platform is Microsoft Foundry, the right approach is not to recreate Foundry. Instead, build the enterprise control plane around Foundry agents. Citadel becomes the pattern that standardizes how agents are registered, secured, evaluated, observed, approved, and retired.

Practical rule: Foundry gives you the agent platform. Citadel gives you the governed operating model for a fleet of agents.

2. Target architecture

- Enterprise AI Governance
- > Security Fabric: Defender, Purview, Entra, SIEM
 - > Agent Identity: managed identities, owners, RBAC, lifecycle
 - > AI Control Plane: tracing, evaluations, dashboards, FinOps
 - > AI Gateway: APIM, quotas, model routing, runtime policy
 - > Microsoft Foundry projects: agents, models, tools, RAG, MCP

3. Out-of-box vs custom build

Capability	Use Microsoft platform	Custom build needed?
Agent development	Microsoft Foundry / Agent Framework	No, but add enterprise templates.
Project isolation	Foundry resource and project model	Configure via IaC.
Tracing	Foundry tracing + App Insights	Normalize and dashboard across agents.
Evaluations	Foundry evaluation	Build golden datasets and release gates.
Gateway	Azure API Management	Build Citadel policy pack.
Identity	Entra ID / managed identity	Build agent identity lifecycle and registry.
Data governance	Purview	Map labels/classification into agent policy.
Security posture	Defender for Cloud	Integrate findings and shadow-agent process.
Approval	Teams, ServiceNow, Logic Apps	Build action-scoped approval service.

4. Highest-value custom components

Custom component	Purpose
Citadel landing zone IaC	Repeatable Foundry, APIM, identity, networking, logging, and security deployment.
AI Gateway policy pack	Runtime controls for authentication, model allowlists, token quotas, routing, and logging.
Agent registry	Enterprise catalog of agents, owners, risk tiers, tools, status, reviews, and evidence.
Tool registry	Inventory of tools, owners, risk levels, approval rules, and allowed agents.
Policy runtime	Before/after model and tool-call checks inside the agent runtime.
Approval service	Human-in-the-loop approval for production remediation.

Custom component	Purpose
AgentOps dashboard	Fleet, runtime, security, evaluation, and FinOps visibility.
Evaluation factory	Golden datasets, automated evaluators, and CI/CD release gates.
Audit/evidence store	Retained evidence for incidents, reviews, and compliance.
Kill switch	Disable agent, tool, environment, or risk tier during unsafe behavior.

5. Module 1 - Foundry Citadel landing zone

Create one landing-zone pattern for dev, test, and prod. The goal is to make every agent project inherit the same security, observability, and governance defaults.

```
citadel-landing-zone/
  infra/
    modules/
      foundry/
      apim/
      observability/
      keyvault/
      identity/
      search/
      storage/
      networking/
    environments/
      dev/
      test/
      prod/
    policies/
      azure-policy/
      apim-policy-fragments/
    pipelines/
      deploy-dev.yml
      deploy-test.yml
      deploy-prod.yml
```

- Provision Foundry resources and projects by domain and environment.
- Configure Application Insights, Log Analytics, diagnostic settings, and retention.
- Create managed identities for agents and platform services.
- Apply network boundaries, private endpoints, Key Vault, and RBAC.
- Ship deployment through Azure DevOps or GitHub Actions.

6. Module 2 - AI Gateway policy pack

Azure API Management should sit in front of model endpoints and governed tool APIs wherever possible. Build reusable policy fragments rather than one-off policies.

```
apim-policy-pack/
  inbound/
    validate-jwt.xml
    enforce-agent-id.xml
    enforce-model-allowlist.xml
    llm-token-limit.xml
    inject-correlation-id.xml
  backend/
    route-by-model-tier.xml
    route-by-environment.xml
    failover-model.xml
  outbound/
    emit-token-metrics.xml
    log-usage.xml
    response-safety-check.xml
  on-error/
    normalize-error.xml
```


audit-policy-failure.xml	
Policy	Reason
Validate user and agent identity	Prevent anonymous or unregistered agent calls.
Model allowlist	Keep production agents on approved models.
Token and quota limit	Prevent cost spikes and runaway loops.
Correlation id injection	Join model calls, tool calls, approvals, and traces.
Request/response logging	Support audit, debugging, and FinOps.

7. Module 3 - Agent registry

The agent registry is the source of truth for enterprise governance. I recommend GitOps YAML as the authoritative record plus an API and portal for visibility.

```
agent:
  id: agt-ops-incident-triage-prod
  name: Incident Triage Agent
  owner_team: platform-ops
  business_owner: head-of-sre
  technical_owner: ops-ai-engineering
  environment: prod
  foundry:
    resource: enterprise-ai-foundry
    project: agentops-prod
  risk:
    tier: 4
    classification: high
  identity:
    managed_identity: mi-agt-ops-incident-triage-prod
  tools:
    allowed:
      - azure_monitor.query_metrics
      - app_insights.query_logs
      - deployment.get_recent
      - service_now.create_ticket
    approval_required:
      - aks.restart_pod
      - app_service.restart
      - deployment.rollback
    blocked:
      - resource.delete
      - keyvault.read_secret
  lifecycle:
    status: approved-prod
    review_frequency: monthly
```

Registry table	Stores
agents	Core agent record, owner, risk tier, environment.
agent_versions	Prompt, tool, model, and deployment version history.
agent_tools	Allowed, blocked, and approval-required tools.
agent_eval_results	Evaluation scores and release gate evidence.
agent_runtime_events	Traces, policy decisions, and usage snapshots.
agent_reviews	Access and lifecycle review evidence.

8. Module 4 - Tool registry

The tool registry is the heart of safe Agentic Ops. Models are risky because they can decide; tools are risky because they can act.

```
tool:
  id: deployment.rollback
```

```

name: Rollback Deployment
owner_team: platform-release-engineering
category: remediation
risk:
  level: high
  blast_radius: service-level
access:
  operation_type: write
  approval_required_in:
    - prod
  allowed_agent_tiers:
    - 4
    - 5
controls:
  requires_change_ticket: true
  requires_human_approval: true
  requires_trace_id: true
audit:
  log_request: true
  log_response: true
  retain_days: 365

```

Risk	Tool examples	Default control
Low	Query metrics, search runbooks	Allow with logging.
Medium	Create ticket, post incident update	Validate inputs and log.
High	Restart pod, scale service, rollback	Require approval in production.
Critical	Delete resource, modify firewall, read secrets	Block by default.

9. Module 5 - Agent policy runtime

The policy runtime intercepts agent actions before and after tool calls. This is needed because APIM controls traffic, but the agent runtime must decide if a specific tool call is allowed for the agent, environment, user, risk, and context.

```

Before tool call:
- Load agent record
- Load tool record
- Check environment and risk
- Check approval requirement
- Check user and agent permission
- Check cost and tool-call limits
- Return ALLOW, BLOCK, or REQUIRE_APPROVAL

After tool result:
- Scan for secrets and PII
- Redact or block unsafe data
- Log result metadata
- Update trust score or violation count

def authorize_tool_call(agent, tool, context):
    if tool.risk == "critical":
        return "BLOCK"
    if context.environment == "prod" and tool.approval_required:
        if not context.approval_id:
            return "REQUIRE_APPROVAL"
    if agent.risk_tier < tool.minimum_agent_tier:
        return "BLOCK"
    if context.daily_cost > agent.daily_budget:
        return "BLOCK"
    return "ALLOW"

```

10. Module 6 - Human approval workflow

Human approval should authorize one exact action. It should not give broad future permission.

```
approval_request:
  approval_id: apr-2026-05-14-001
  agent_id: agt-ops-incident-triage-prod
  tool: deployment.rollback
  environment: prod
  service: checkout-service
  target_version: v2.17.2
  reason: Latency spike started after v2.17.3 deployment.
  trace_id: trace-abc-123
  risk_level: high
  expires_in_minutes: 15
  approver_group: sre-incident-commanders
```

- Use Teams Adaptive Cards for fast pilot approvals.
- Integrate ServiceNow for ITSM-heavy environments.
- Persist approval, rejection, expiry, execution result, approver identity, and trace id.
- Require reapproval if tool arguments change.

11. Module 7 - AgentOps observability

Foundry and Application Insights give the raw telemetry. Your custom build should normalize telemetry into a single event model and dashboard across agents.

```
normalized_event:
  timestamp: 2026-05-14T10:15:00Z
  trace_id: trace-abc-123
  agent_id: agt-ops-incident-triage-prod
  agent_version: 1.4.2
  user_id: user-123
  environment: prod
  event_type: tool_call
  tool_name: azure_monitor.query_metrics
  tool_risk: low
  status: success
  latency_ms: 875
  input_tokens: 3200
  output_tokens: 600
  cost_usd: 0.08
  policy_decision: ALLOW
```

Dashboard	Must show
Fleet	Agents by risk, owner, environment, status, review due.
Runtime	Runs, task success, tool-call success, latency, errors.
Policy	Blocked calls, approvals, violations, unsafe attempts.
Eval	Pass rate, regressions, failed scenarios, release gates.
FinOps	Cost by agent, model, tool, team, and environment.

12. Module 8 - Evaluation and release gates

A Foundry-first Citadel implementation should make evaluation part of CI/CD. Every agent version should run against a golden dataset before promotion.

```
agent-evaluation-factory/
  datasets/
    incident-triage/
      latency-spike.jsonl
      prompt-injection.jsonl
      dependency-failure.jsonl
      false-positive-alert.jsonl
  evaluators/
    tool_selection.py
    approval_compliance.py
```

```
groundedness.py
remediation_safety.py
cost_efficiency.py
pipelines/
  run-pr-evals.yml
  nightly-regression.yml
  pre-prod-gate.yml
```

Gate	Threshold
Task success	>= 90%
Correct tool selection	>= 95%
Approval compliance	100%
Unsafe tool calls	0
PII leakage	0
Cost per eval case	Within budget threshold

13. Module 9 - Agentic Ops workbench

The workbench is the UI where SRE, platform, security, risk, and compliance teams see the state of the agent fleet.

- Agent Catalog: agents by domain, owner, risk, environment, status.
- Agent Detail: purpose, identity, tools, eval history, traces, versions.
- Tool Catalog: risk level, owner, allowed agents, approval rules.
- Approval Queue: pending high-risk actions with trace evidence.
- Incident View: investigation trace, recommendations, approvals, and results.
- Eval Dashboard: regression trend and failed test cases.
- Security View: prompt-injection attempts and sensitive-data events.
- FinOps View: cost by agent, model, tool, team, and environment.

14. Module 10 - Agent incident response and kill switch

Agents need their own incident response model. Unsafe behavior should generate an operational/security event and may automatically reduce the agent to read-only mode.

```
When policy violation occurs:
1. Create security/ops event
2. Attach trace and policy decision
3. Notify agent owner and security team
4. Disable high-risk tools if severity is high
5. Open incident ticket
6. Add scenario to evaluation backlog

Kill switch levels:
- disable_agent
- disable_agent_in_prod
- disable_tool_for_agent
- switch_agent_to_read_only
- block_model_route
- disable_all_tier_4_agents
```

15. Implementation backlog

Epic	Priority P0 stories
Foundation	Foundry IaC, APIM, App Insights, Key Vault, managed identities, RBAC.
AI Gateway	Validate agent id, model allowlist, token limits, cost metrics, audit logging.
Agent Registry	Schema, API, GitOps validation, lifecycle states, ownership tracking.
Tool Registry	Tool schema, risk classification, approval metadata, wrapper SDK.
Policy Runtime	Before-tool-call interceptor, registry lookup, approval integration, audit events.
Approval	Approval schema, API, expiry, evidence logging, Teams or ServiceNow channel.
Observability	Foundry tracing, normalized events, AgentOps dashboard, policy dashboard.
Evaluations	Golden dataset, tool-selection evaluator, approval-compliance evaluator, CI/CD gate.
Security	Defender, Purview, Entra groups, SIEM events, shadow-agent process.

16. Recommended repository structure

```
citadel-foundry-platform/  
  infra/  
    modules/  
    environments/  
  platform-services/  
    agent-registry-api/  
    tool-registry-api/  
    policy-runtime/  
    approval-service/  
    audit-service/  
    finops-service/  
  agents/  
    incident-triage-agent/  
      agent.yaml  
      tools.yaml  
      evals/  
      src/  
    runbook-assistant/  
      agent.yaml  
      tools.yaml  
      evals/  
      src/  
  policies/  
    gateway/apim/  
    runtime/  
  evals/  
    datasets/  
    evaluators/  
    pipelines/  
  observability/  
    workbooks/  
    dashboards/  
    kql/  
    alerts/  
  docs/  
    operating-model.md  
    agent-onboarding.md  
    production-readiness-checklist.md  
    incident-response.md
```

17. First five sprints

Sprint	Goal	Deliverables
1	Foundation	Foundry project pattern, APIM, App Insights, identities, schemas.
2	First Agentic Ops agent	Incident Triage Agent with read-only tools and traces.

Sprint	Goal	Deliverables
3	Policy and approval	Policy runtime, approval workflow, audit events.
4	Evaluations	Golden dataset, evaluators, CI/CD release gate.
5	Security and scale	Purview, Defender, kill switch, cost dashboard, review workflow.

18. Minimum viable Citadel for Microsoft Foundry

Microsoft Foundry

- > Agentic Ops project
- > Incident Triage Agent

Azure API Management

- > AI Gateway policies

Azure App Service or Container Apps

- > Agent Registry API
- > Tool Registry API
- > Policy Runtime API
- > Approval Service
- > AgentOps Portal

Azure Monitor

- > Application Insights
- > Log Analytics

Microsoft Entra

- > User auth, agent managed identities, approval groups

Microsoft Purview and Defender

- > Data classification and security posture signals

19. Build priority

Build in this order: Agent registry, Tool registry, Policy runtime, Approval service, AgentOps dashboard, Evaluation pipeline, APIM policy pack, Audit store, Lifecycle workflow, Kill switch. This sequence gives you safe production control before you expand to multiple agents or autonomous remediation.

References used for the original answer

- Azure Samples - Foundry Citadel Platform: <https://github.com/Azure-Samples/foundry-citadel-platform>
- Azure AI Foundry architecture concepts:
<https://learn.microsoft.com/en-us/azure/foundry/concepts/architecture>
- Microsoft Foundry overview: <https://learn.microsoft.com/en-us/azure/foundry/what-is-foundry>
- Azure API Management LLM token limit policy:
<https://learn.microsoft.com/en-us/azure/api-management/llm-token-limit-policy>
- Azure AI Foundry tracing setup:
<https://learn.microsoft.com/en-us/azure/foundry/observability/how-to/trace-agent-setup>
- Azure AI Foundry trace concepts:
<https://learn.microsoft.com/en-us/azure/foundry/observability/concepts/trace-agent-concept>
- Microsoft Agent Framework overview: <https://learn.microsoft.com/en-us/agent-framework/overview/>
- Evaluate agents in Azure AI Foundry:
<https://learn.microsoft.com/en-us/azure/foundry/observability/how-to/evaluate-agent>
- Microsoft AI Agent Evals GitHub Action: <https://github.com/microsoft/ai-agent-evals>

- Microsoft Defender for Cloud AI security best practices:

<https://learn.microsoft.com/en-us/azure/security/fundamentals/ai-security-best-practices>

- Microsoft Purview Data Security Posture Management:

<https://learn.microsoft.com/en-us/purview/data-security-posture-management-learn-about>

Part 3: Hub-and-Spoke Landing Zone for Microsoft Foundry Citadel

Detailed Azure service mapping and implementation guidance for Agentic Ops.

Hub-and-spoke landing zone: concept and Azure implementation mapping

A hub-and-spoke landing zone is an Azure architecture pattern where shared platform services live in a central hub, and business/application workloads live in separate spokes.

Simple mental model:

Hub = airport control tower + security checkpoint + shared utilities
Spoke = individual airline terminals / business workloads

For this Citadel and Microsoft Foundry context:

Hub = Citadel / Foundry platform landing zone
Spoke = each AI workload, agent, domain, or business-unit Foundry project

1. Why hub-and-spoke is useful for Citadel and Agentic Ops

Agentic Ops requires central control without blocking team autonomy. You want platform-level controls such as identity, networking, private access, AI gateway, model access policies, token/cost controls, monitoring, security posture, data governance, audit logging, and approval workflows.

At the same time, each application team should be able to build its own agents, for example incident triage agents, runbook assistants, change-risk advisors, cloud cost optimization agents, service reliability agents, and knowledge/RAG assistants.

Hub-and-spoke gives you both: centralized governance in the hub plus decentralized innovation in the spokes.

2. High-level Azure layout

```
Azure Tenant
|-- Management Group: Platform
|   |-- Subscription: Connectivity Hub
|   |   |-- Hub VNet
|   |   |-- Azure Firewall
|   |   |-- VPN / ExpressRoute Gateway
|   |   |-- Private DNS Zones
|   |   |-- Azure Bastion
|   |   |-- DDoS Protection
|   |-- Subscription: Management / Observability
|   |   |-- Log Analytics Workspace
|   |   |-- Application Insights
|   |   |-- Azure Monitor
|   |   |-- Microsoft Sentinel
|   |-- Subscription: Identity / Security
|   |   |-- Microsoft Entra ID integration
```



```

| | | -- Key Vault
| | | -- Defender for Cloud
| | | -- Purview
| | |
| | | -- Subscription: AI Platform Hub / Citadel Hub
| | | -- Azure API Management
| | | -- Foundry Hub / Foundry resource
| | | -- Agent Registry
| | | -- Tool Registry
| | | -- Policy Runtime
| | | -- Approval Service
| | | -- AgentOps Portal
| | |
| | | -- Management Group: Landing Zones / Workloads
| | |
| | | -- Subscription: AgentOps Spoke - Dev/Test
| | | -- Subscription: AgentOps Spoke - Prod
| | | -- Subscription: Business App AI Spoke

```

3. Hub vs spoke responsibilities

Hub responsibilities

- Central network connectivity
- Internet egress control
- On-prem connectivity
- Private DNS
- Shared firewall
- Shared API gateway
- AI gateway policies
- Central observability
- Security monitoring
- Agent registry
- Tool registry
- Approval workflow
- Policy runtime services
- Cost and audit reporting

Spoke responsibilities

- Agent code
- Domain tools
- Domain RAG pipelines
- Domain data connections
- Workload compute
- Workload-specific storage
- Team-level CI/CD
- Domain-specific evaluations

Shared responsibility

Platform team:
 Defines guardrails, shared services, policies, networking, observability.

Workload team:
 Builds agents and tools inside the guardrails.

Security/risk team:
 Defines risk tiers, data controls, approval rules, evidence requirements.

Operations/SRE:
 Defines Agentic Ops runbooks, remediation boundaries, escalation rules.

4. Azure service mapping by landing-zone function

4.1 Management groups and subscriptions

Landing-zone function	Azure service
Enterprise hierarchy	Azure Management Groups
Subscription organization	Azure Subscriptions
Policy inheritance	Azure Policy
RBAC inheritance	Azure RBAC
Cost boundaries	Azure Cost Management
Environment isolation	Separate dev/test/prod subscriptions

Recommended hierarchy:

```

Tenant Root Group
|
|-- Platform
|   |-- Connectivity
|   |-- Management
|   |-- Identity-Security
|   |-- AI-Platform-Hub
|
|-- Landing-Zones
|   |-- AgentOps-Dev
|   |-- AgentOps-Test
|   |-- AgentOps-Prod
|   |-- Finance-AI
|   |-- HR-AI
|   |-- Customer-Service-AI
|
|-- Sandbox
|   |-- AI-Experiments

```

4.2 Connectivity hub

The connectivity hub is the core network platform.

Requirement	Azure service
Central network	Azure Virtual Network
Spoke connectivity	VNet Peering or Azure Virtual Network Manager
Central firewall	Azure Firewall Premium
Internet egress control	Azure Firewall + Route Tables
On-prem connectivity	ExpressRoute / VPN Gateway
Secure admin access	Azure Bastion
DNS for private services	Azure Private DNS Zones

Requirement	Azure service
DDoS protection	Azure DDoS Network Protection
Network segmentation	NSG / ASG
Traffic routing	User Defined Routes
Global routing option	Azure Virtual WAN

Typical hub network:

Hub VNet: 10.0.0.0/16

Subnets:

- AzureFirewallSubnet
- GatewaySubnet
- AzureBastionSubnet
- PrivateEndpointSubnet
- SharedServicesSubnet
- APIMSubnet

Spoke networks:

AgentOps-Prod-Spoke VNet: 10.10.0.0/16

Finance-AI-Spoke VNet: 10.20.0.0/16

Support-AI-Spoke VNet: 10.30.0.0/16

Traffic flow:

Spoke agent workload

- > Route table sends outbound traffic to hub firewall
- > Azure Firewall inspects and allows/blocks
- > Approved target: Foundry, Azure Monitor, ServiceNow, internal APIs

4.3 Private connectivity for AI workloads

For production Agentic Ops, prefer private networking.

Requirement	Azure service
Private access to PaaS	Azure Private Link
Private service endpoint	Private Endpoint
Name resolution	Azure Private DNS Zones
Disable public access	PaaS firewall / public network disabled
Secure service-to-service calls	Managed identity + private endpoint

Recommended private endpoints:

- Azure AI Foundry / Azure AI services where supported
- Azure OpenAI / model endpoints
- Azure AI Search
- Storage Account
- Key Vault
- Cosmos DB / Azure SQL
- Log Analytics ingestion where applicable
- Container Registry

Private DNS zones commonly needed:

```
privatelink.openai.azure.com
privatelink.search.windows.net
privatelink.blob.core.windows.net
privatelink.vaultcore.azure.net
privatelink.documents.azure.com
privatelink.azurewebsites.net
```

5. Citadel-specific hub-and-spoke mapping

For a Microsoft Foundry Citadel platform, use the hub for control and the spokes for agent execution.

5.1 Citadel Hub

```
AI Platform Hub Subscription
|
|-- Azure API Management
|   |-- AI Gateway
|
|-- Microsoft Foundry resource / central AI hub
|   |-- Shared models, shared connections, governance baseline
|
|-- Agent Registry Service
|-- Tool Registry Service
|-- Policy Runtime Service
|-- Approval Service
|-- Audit Evidence Store
|-- AgentOps Portal
|-- Shared Observability
```

5.2 Citadel Spokes

```
AgentOps Production Spoke
|
|-- Foundry Project: agentops-prod
|-- Incident Triage Agent
|-- Change Risk Agent
|-- Runbook RAG Agent
|-- Azure AI Search indexes
|-- Storage for grounding documents
|-- App Service / Container Apps for agent runtime
|-- Workload-specific APIs
|-- Private Endpoints
|-- Managed identities
```

6. Mapping Citadel layers to Azure services

Layer 1: Governance Hub / AI Gateway

Citadel capability	Azure service
Central AI gateway	Azure API Management
Token throttling	APIM llm-token-limit policy
API authentication	APIM + Microsoft Entra ID
Model allowlist	APIM policy
Request validation	APIM policy
Response filtering	APIM policy + custom service
Cost attribution	APIM logs + Azure Monitor + Cost Management
Correlation ID	APIM policy

Citadel capability	Azure service
Central logging	APIM diagnostic settings to Log Analytics

Example gateway flow:

```

Agent call
-> APIM validates identity and agent ID
-> APIM checks model allowlist
-> APIM applies token limit and quota
-> APIM forwards to approved model endpoint
-> APIM emits telemetry and audit log

```

Layer 2: AI Control Plane / AgentOps

Citadel capability	Azure service
Tracing	Azure AI Foundry tracing / OpenTelemetry
Application telemetry	Application Insights
Central log analytics	Log Analytics Workspace
Dashboards	Azure Monitor Workbooks / Power BI
Alerts	Azure Monitor Alerts
SIEM	Microsoft Sentinel
Eval results storage	Cosmos DB / Azure SQL / Storage
AgentOps portal	Azure App Service / Container Apps
Event processing	Event Grid / Event Hub / Azure Functions

Recommended data flow:

```

Foundry agent trace
-> Application Insights
-> Log Analytics
-> Azure Monitor Workbook / Power BI
-> Alerts to Teams / ServiceNow / Sentinel

```

Layer 3: Agent Identity and Lifecycle

Citadel capability	Azure service
Human identity	Microsoft Entra ID
Agent/service identity	Managed Identity / Entra application
Access control	Azure RBAC
Group-based authorization	Entra security groups
Privileged access	Entra Privileged Identity Management
Secrets	Azure Key Vault
Agent catalog	Custom Agent Registry
Lifecycle workflow	Logic Apps / Azure Functions / DevOps pipeline
Access reviews	Entra ID Governance

Recommended identity pattern:

One managed identity per production agent or agent group

```

Example:
mi-agentops-incident-triage-prod
mi-agentops-runbook-reader-prod

```

Good pattern:

User asks agent to investigate
 -> Agent executes with agent managed identity
 -> High-risk action requires human approval
 -> Audit records both agent identity and approving human identity

Layer 4: Security Fabric

Citadel capability	Azure service
Cloud security posture	Microsoft Defender for Cloud
AI workload discovery/posture	Defender for Cloud AI security posture
Data discovery/classification	Microsoft Purview
Sensitive data controls	Purview sensitivity labels / DLP
SIEM/SOAR	Microsoft Sentinel
Identity threat protection	Entra ID Protection
Secrets protection	Key Vault + Defender
Network threat inspection	Azure Firewall Premium
Audit retention	Log Analytics archive / Storage immutable policies

Security event examples:

- Agent attempted blocked production action
- Agent tried to call unapproved external endpoint
- Tool response contained secrets
- Prompt injection detected in retrieved logs
- Agent exceeded token budget
- Unregistered agent called the gateway

7. Concrete implementation: subscriptions and resource groups**Platform subscriptions**

```

sub-connectivity-prod
|
|-- rg-network-hub-prod
|   |-- vnet-hub-prod
|   |-- azure-firewall-prod
|   |-- route-tables
|   |-- vpn-gateway / expressroute-gateway
|   |-- azure-bastion
|   |-- ddos-plan
|
|-- rg-private-dns-prod
|   |-- privatelink.openai.azure.com
|   |-- privatelink.search.windows.net
|   |-- privatelink.blob.core.windows.net
|   |-- privatelink.vaultcore.azure.net
|   |-- privatelink.azurewebsites.net
|
sub-management-prod
|

```

```

|-- rg-monitoring-prod
|   |-- log-analytics-enterprise-prod
|   |-- appinsights-citadel-prod
|   |-- azure-monitor-workbooks
|   |-- action-groups
|
|-- rg-sentinel-prod
|   |-- microsoft-sentinel
|
sub-ai-platform-prod
|
|-- rg-citadel-hub-prod
|   |-- apim-ai-gateway-prod
|   |-- foundry-enterprise-prod
|   |-- app-agent-registry-prod
|   |-- app-tool-registry-prod
|   |-- app-policy-runtime-prod
|   |-- app-approval-service-prod
|   |-- app-agentops-portal-prod
|   |-- cosmos-citadel-prod
|   |-- kv-citadel-prod
|   |-- st-citadelauditprod

```

AgentOps production spoke

```

sub-agentops-prod
|
|-- rg-agentops-network-prod
|   |-- vnet-agentops-prod
|   |-- nsg-agentops-prod
|   |-- route-table-to-hub-firewall
|   |-- private-endpoints
|
|-- rg-agentops-foundry-prod
|   |-- foundry-project-agentops-prod
|   |-- agent-incident-triage-prod
|   |-- agent-runbook-assistant-prod
|   |-- agent-change-risk-prod
|
|-- rg-agentops-data-prod
|   |-- azure-ai-search-agentops-prod
|   |-- storage-runbooks-prod
|   |-- cosmos-agent-state-prod
|
|-- rg-agentops-runtime-prod
|   |-- appsvc-agent-runtime-prod
|   |-- container-apps-agent-tools-prod
|   |-- managed-identity-agentops-prod
|   |-- keyvault-agentops-prod

```

8. Network flow for Foundry/Citadel

8.1 Model call flow

Agent runtime in spoke

- > Azure API Management in hub
- > Policy checks: caller identity, agent ID, model allowlist, token budget, environment, correlation ID
- > Approved model endpoint / Foundry model deployment
- > Response returns through APIM
- > Telemetry to Log Analytics

8.2 Tool call flow

Agent runtime in spoke

- > Policy middleware checks local context
- > Tool Registry in hub
- > Policy Runtime in hub
- > Decision: ALLOW / BLOCK / REQUIRE_APPROVAL
- > If ALLOW: call tool API
- > If REQUIRE_APPROVAL: Approval Service creates approval request

8.3 High-risk Agentic Ops action flow

Incident Triage Agent recommends rollback
-> Policy Runtime sees tool = deployment.rollback, environment = prod, risk = high
-> Approval Service sends request to Teams / ServiceNow
-> Incident commander approves
-> Policy Runtime issues approval token
-> Agent calls rollback tool with approval token
-> Audit event stored

9. Azure services for each Citadel custom component

Agent Registry

Need	Azure implementation
API	Azure App Service / Container Apps
Data store	Cosmos DB or Azure SQL
Source of truth	Git repository YAML + CI validation
Authentication	Entra ID
Authorization	Entra groups + app roles
Audit	Log Analytics + Storage
UI	Static Web App / App Service

Tool Registry

Need	Azure implementation
Tool metadata	Cosmos DB / Azure SQL
Tool schema validation	Azure Functions / App Service
API documentation	API Management developer portal
Tool wrappers	Container Apps / Functions
Risk classification	Registry metadata + policy engine
Secrets	Key Vault

Policy Runtime

Need	Azure implementation
Runtime authorization API	Container Apps / App Service
Rules engine	Custom rules / OPA container / Azure Functions
Fast cache	Azure Cache for Redis
Event logging	Event Hub / Log Analytics
Policy source	Git repo + CI/CD
High availability	Zone-redundant App Service / Container Apps

Approval Service

Need	Azure implementation
Workflow engine	Logic Apps / Durable Functions
Teams approval	Teams Adaptive Cards
ITSM integration	ServiceNow connector / custom API
State store	Cosmos DB / Azure SQL
Expiry handling	Durable Functions timer
Audit	Log Analytics + immutable Storage

AgentOps Portal

Need	Azure implementation
Web UI	Azure Static Web Apps / App Service
Backend	App Service / Container Apps
Dashboards	Azure Monitor Workbooks / Power BI
Auth	Entra ID
Data	Cosmos DB / Log Analytics queries
Alerts	Azure Monitor Action Groups

10. Example Azure policy and governance mapping

Governance requirement: no public AI endpoints in production

Azure services:

- Azure Policy
- Private Endpoint
- PaaS public network access disabled
- Defender for Cloud recommendations

Policy behavior:

Deny production deployment if:

- Storage account allows public access
- Key Vault public access is enabled
- Search service public access is enabled
- AI service does not use private endpoint

Governance requirement: all production agents must be registered

Azure services:

- Agent Registry
- APIM policy
- Azure Monitor

Runtime behavior:

APIM receives request

- > Checks x-agent-id header
- > Calls Agent Registry or cached registry
- > If agent is unknown: block request

Governance requirement: high-risk tools need approval

Azure services:

- Tool Registry
- Policy Runtime
- Approval Service
- Teams / ServiceNow
- Audit store

Runtime behavior:

- Agent requests deployment.rollback
 - > Tool Registry says risk = high
 - > Policy Runtime says production high-risk action requires approval
 - > Approval Service creates approval
 - > Tool call blocked until approval token exists

11. Practical hub-and-spoke implementation steps

Step 1: Define management group hierarchy

```
Tenant Root
|-- Platform
|   |-- Connectivity
|   |-- Management
|   |-- Security
|   |-- AI-Platform
|-- Landing-Zones
|   |-- AgentOps
|   |-- Business-AI
|   |-- Sandbox
```

Apply Azure Policy at the right levels:

```
Platform:
  Enforce diagnostics
  Enforce Defender
  Enforce resource tagging

Landing-Zones:
  Enforce private endpoints
  Restrict regions
  Restrict public IPs
  Enforce allowed SKUs
  Require managed identity
```

Step 2: Build the connectivity hub

Deploy hub VNet, Azure Firewall Premium, Firewall Policy, route tables, VPN/ExpressRoute if needed, Bastion, Private DNS Zones, and DDoS protection.

Configure spoke routing:

```
0.0.0.0/0 -> Azure Firewall private IP
On-prem CIDR -> ExpressRoute/VPN gateway
Hub shared services CIDR -> VNet peering
```

Step 3: Create the AI Platform Hub

Deploy Azure API Management, Foundry resource/hub, Agent Registry API, Tool Registry API, Policy Runtime, Approval Service, AgentOps Portal, Key Vault, Cosmos DB/Azure SQL, and Storage for audit evidence.

For Citadel, place APIM in the AI Platform Hub subscription with network integration to the hub VNet and centralized routing through the connectivity hub.

Step 4: Create first spoke: AgentOps

Deploy spoke VNet, VNet peering to hub, route table to firewall, private endpoints, Foundry project, agent runtime, Azure AI Search, Storage, App Service/Container Apps, and managed identities.

Register the spoke in Azure Virtual Network Manager if used, Private DNS links, Azure Policy scope, monitoring diagnostic settings, and the Agent Registry.

Step 5: Implement the AI Gateway

APIM policies should enforce JWT validation, Agent ID validation, model allowlist, token rate limit, token quota, environment restrictions, correlation ID, and logging.

Conceptual APIM policy flow:

```
<inbound>
  <validate-jwt />
  <set-header name="x-correlation-id" />
  <send-request mode="new">
    <!-- Validate x-agent-id against Agent Registry -->
  </send-request>
  <llm-token-limit
    counter-key="@context.Request.Headers.GetValueOrDefault('x-agent-id','unknown')"
    tokens-per-minute="200000"
    token-quota="10000000"
    token-quota-period="Daily"
    estimate-prompt-tokens="true" />
  <choose>
    <!-- Block non-approved model names -->
  </choose>
</inbound>
```

Step 6: Implement private endpoints and DNS

For each spoke PaaS service:

- Create private endpoint in spoke VNet
- Disable public network access where possible
- Link private DNS zone from hub to spoke VNet
- Validate name resolution from spoke runtime

Example:

```
Agent runtime calls: https://mysearch.search.windows.net
DNS resolves to: private endpoint IP in spoke VNet
Traffic stays private: Agent runtime -> Private Endpoint -> Azure AI Search
```

Step 7: Centralize observability

Every component sends logs to central Log Analytics:

- APIM diagnostics -> Log Analytics
- Agent runtime logs -> Application Insights
- Foundry traces -> Application Insights / Azure Monitor
- Policy decisions -> Log Analytics
- Approval events -> Log Analytics
- Tool calls -> Log Analytics
- Firewall logs -> Log Analytics
- Defender alerts -> Sentinel

12. Example: Agentic Ops hub-and-spoke in action

Scenario:

Investigate checkout latency in production.

Execution path:

1. User authenticates with Entra ID.
2. Request goes to Incident Triage Agent in AgentOps Prod Spoke.
3. Agent calls Azure Monitor query tool.
4. Policy middleware checks agent registration, tool permission, and read-only operation. Decision: ALLOW.
5. Agent calls deployment history tool.
6. Agent detects latency started after deployment v2.17.3.
7. Agent recommends rollback.
8. Agent attempts rollback tool.
9. Policy Runtime sees production + high risk + missing approval. Decision: REQUIRE_APPROVAL.
10. Approval Service sends request to incident commander.
11. Human approves.
12. Agent executes rollback tool with approval token.
13. All actions are traced and stored.

Azure service path:

Entra ID
-> Agent Runtime on App Service / Container Apps
-> Policy Runtime on App Service / Container Apps
-> Tool Registry on Cosmos DB / Azure SQL
-> Approval Service on Logic Apps / Durable Functions
-> Azure Monitor / Log Analytics
-> Deployment API
-> Application Insights trace
-> Sentinel if security event

13. Recommended hub-and-spoke design for this Citadel implementation

Minimal production version:

Subscriptions:
1. sub-connectivity-prod
2. sub-management-prod
3. sub-ai-platform-prod
4. sub-agentops-prod
5. sub-agentops-nonprod

Hub services:

Connectivity Hub:

- Hub VNet
- Azure Firewall
- Private DNS
- Bastion
- ExpressRoute/VPN if needed

Management Hub:

- Log Analytics
- Application Insights
- Azure Monitor
- Sentinel

AI Platform Hub:

- Foundry resource
- APIM AI Gateway
- Agent Registry
- Tool Registry
- Policy Runtime
- Approval Service
- AgentOps Portal
- Key Vault

Spoke services:

AgentOps Prod Spoke:

- Foundry project
- Agent runtime
- Azure AI Search
- Storage
- App Service / Container Apps
- Private endpoints
- Managed identities

14. Important design choices

Choice 1: One hub or multiple hubs?

Use one hub for a single region, moderate scale, a central platform team, and simpler governance. Use multiple regional hubs when you need multiple Azure regions, data residency, latency-sensitive workloads, or regional failover.

Choice 2: VNet peering or Azure Virtual WAN?

Use VNet peering when you want more control, fewer regions, or custom firewall/NVA patterns. Use Azure Virtual WAN when you need Microsoft-managed global hub connectivity, many branches/regions, or large-scale network simplification.

Choice 3: Central Foundry resource or per-spoke Foundry resources?

Recommended hybrid:

Central Foundry resource:

- Shared governance
- Shared approved model deployments
- Shared platform-level connections

Spoke Foundry projects/resources:

- Domain agents
- Team isolation
- Data boundary
- Environment separation

For strict isolation, use separate Foundry resources per spoke/environment. For easier sharing, use one Foundry resource with separate projects, but confirm your organization's isolation, networking, and compliance requirements.

15. Landing-zone implementation checklist

Platform foundation

- Management group hierarchy created
- Subscriptions assigned
- Naming convention defined
- Tagging standard enforced
- Azure Policy baseline applied
- RBAC model defined
- Cost management budgets created

Connectivity hub

- Hub VNet deployed
- Azure Firewall deployed
- Firewall policies configured
- Route tables configured
- Spoke VNet peering configured
- Private DNS zones created
- Private DNS linked to spokes
- ExpressRoute/VPN configured if needed

AI Platform Hub

- Foundry resource deployed
- APIM AI Gateway deployed
- Agent Registry deployed
- Tool Registry deployed
- Policy Runtime deployed
- Approval Service deployed
- AgentOps Portal deployed
- Key Vault deployed
- Audit storage deployed

Spoke

- Spoke VNet deployed
- Route table points to hub firewall

- Foundry project created
- Agent runtime deployed
- Managed identities assigned
- Private endpoints created
- Public network access disabled where possible
- Diagnostics enabled

Agentic Ops controls

- Agents registered
- Tools registered
- Tool risk levels assigned
- High-risk actions require approval
- Tracing enabled
- Eval pipeline enabled
- Token limits enabled
- Kill switch implemented
- Audit evidence retained

16. Final mental model

For a Microsoft Foundry + Citadel platform:

Hub = control plane
Spoke = workload execution plane

More specifically:

Hub:
Guardrails, identity, network, gateway, monitoring, audit, policy, approval

Spoke:
Agents, tools, RAG, workload APIs, domain data, business logic

The hub-and-spoke landing zone lets you scale from one useful agent to a governed enterprise fleet of agents without every team reinventing security, networking, identity, observability, cost control, and approval workflows.